

Chapter 6. Overview of the Financial Data Engineering Lifecycle

As a financial data engineer, navigating the multitude, diversity, and complexity of available technological options can be overwhelming. Without a systematic approach in mind, this complexity may lead to chaotic situations and accumulating costly technical debt. Therefore, this chapter introduces a structured approach to financial data engineering, organizing its components into a layered architecture called the *financial data engineering lifecycle* (FDEL). This framework draws inspiration from the foundational [work of Joe Reis and Matt Housley](#) on data engineering lifecycles.

In this chapter, I will introduce the FDEL and outline its four layers: ingestion, storage, transformation and delivery, and monitoring. Following this, I will discuss specific criteria that financial data engineers can consider when selecting technologies to support the FDEL. Please note that since there is so much information to cover, I'll save the details of each FDEL layer for Chapters [7](#) through [10](#).

Financial Data Engineering Lifecycle Defined

Data engineering is a fast-moving and continuously evolving field. However, if there is one constant that characterizes and defines the job of a data engineer, it is the fact that it revolves around systems that perform a series of actions for the extraction, transformation, storage, and consumption of data.

To move beyond the simple view of data engineering as merely a series of data-related tasks, authors Joe Reis and Matt Housley [introduced the *Data Engineering Lifecycle \(DEL\)*](#). This concept offers a structured framework that formalizes the various stages of data engineering. According to the authors, the DEL “comprises stages that turn raw data ingredients into a useful end product, ready for consumption by analysts, data scientists, ML engineers, and others.”

NOTE

It is essential to differentiate between the data lifecycle and the data engineering lifecycle. While they are related, the data lifecycle is a conceptual model that describes the stages data goes through from creation to archival or deletion. On the other hand, the data engineering lifecycle is a practical framework that outlines the processes, patterns, and tools used to manage data throughout its lifecycle, ultimately delivering a final product to data consumers.

The DEL can be adapted to incorporate several processes or stages depending on how complicated the data lifecycle and business needs are. In their original work, Joe Reis and Matt Housley divided the DEL into five stages: data generation, storage, ingestion, transformation, and serving.

When applied to the financial domain, the data engineering lifecycle needs to be adapted to account for a number of domain-specific elements. These originate from the strict data governance and regulatory requirements for financial institutions, legal and content licensing constraints, the complexity of the financial data landscape, industry-specific software standards and protocols, and the unique demands for low latency, high throughput, and other performance constraints that characterize financial operations.

To offer a domain-focused perspective that incorporates these considerations, this book introduces the *financial data engineering lifecycle* (FDEL), a layered framework¹ consisting of four layers: *ingestion layer*, *storage layer*, *transformation and delivery layer*, and *monitoring layer*. [Figure 6-1](#) illustrates this framework.

In the ingestion layer, data engineers design and implement an infrastructure for handling the generation and reception of data coming from different sources and in different formats, volumes, and frequencies. This layer is quite critical, as errors or performance bottlenecks in ingestion are likely to propagate downstream and impact the entire FDEL. In [Chapter 7](#), we will explore the ingestion layer in depth, covering various ingestion processes, patterns, technologies, and formats.

Following is the storage layer, where the focus is on selecting and optimizing data storage models and technologies to meet various business requirements. Choosing the right data storage system for a given problem is one of the most important decisions throughout the FDEL. A poor data storage choice can very easily translate into bottlenecks, degraded performance, and rocketing costs, which in turn can jeopardize product development and analytical capabilities. [Chapter 8](#) will detail the varieties of data storage models and illustrate their data modeling principles, internal features, and financial use cases.

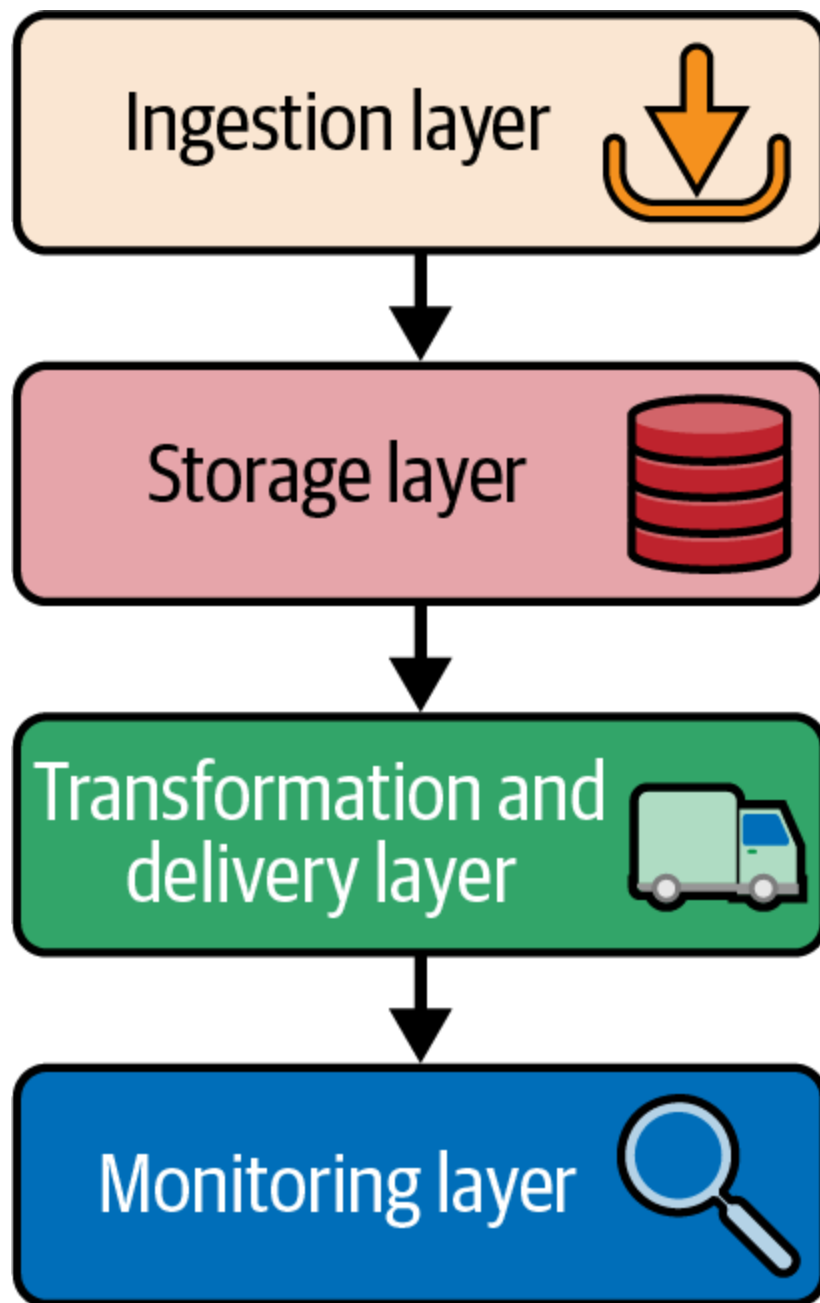


Figure 6-1. Layers of the financial data engineering lifecycle

Next is the transformation and delivery layer, which performs a number of business-defined transformations and computations that produce high-quality data that is ready for consumption by its intended data consumers. In [Chapter 9](#), I will thoroughly explore this layer, examining the key types of transformations applicable to financial data, the computational demands of different transformation processes, and the various mechanisms for data delivery.

Finally, the monitoring layer is there to make sure that issues related to data processing, data quality, performance, costs, bugs, and analytical errors are all monitored and tracked to allow efficient and timely fixes.

[Chapter 10](#) will be dedicated to this layer.

The FDEL framework will provide several advantages if correctly applied. You may benefit greatly, for instance, in terms of modularity and separation of responsibility. The larger the firm is, the larger and more complex its data engineering processes are going to become. This suggests that, from an organizational perspective, it could be feasible to assign distinct teams to each FDEL layer. For example, a team may be dedicated to optimizing and securing the ingestion layer, another for ensuring the performance, scalability, and security of the storage layer, and so on. It is important to note that the FDEL is an *iterative* process where feedback and requirements from one layer may impact the design, constraints, and implementation of another layer.

Over the next four chapters, I will detail these layers, highlighting the domain-specific challenges they present in financial markets, and offering concrete examples to provide deeper insights.

Criteria for Building the Financial Data Engineering Stack

When implementing the FDEL, it's essential to choose the best tools and technologies to design and architect each layer, i.e., the technological stack. This is no small feat. If you want to learn about data engineering technologies and you do a quick search, you will find hundreds of different tools and frameworks. This makes it hard to know where to start since there is practically a jungle of tools to choose from. To make it more complex, these diverse tools can be used together and sometimes compete to process financial data throughout its lifecycle.

In this section, I will outline six criteria that financial institutions can employ to inform their technological decisions. These criteria serve as a fundamental and general guideline but are not exhaustive. Depending on your organization's unique needs, you may need to identify additional criteria. Furthermore, in the following five chapters, I will discuss more granular criteria that are tailored to each layer of the FDEL.

Criterion 1: Open Source Versus Commercial Software

A crucial and highly debated decision in financial institutions concerns the choice between proprietary commercial software and open source software. Proprietary commercial software is distributed under a purchased license that restricts user access to the source code. On the other hand, open source software is typically distributed with a license to access, use, modify, sell, and distribute the source code.

The main advantages of commercial software include the following:

Vendor accountability

The software provider ensures frequent updates, bug fixes, security, and customer support. This is typically ensured via a *Service-Level Agreement* (SLA), which is a guarantee by the vendor to commit to a certain level of quality, availability, and performance of the offered service. Vendors might need to pay a penalty if their SLA is not met. SLAs and security are two of the most important factors behind the widespread reliance on proprietary software by the financial sector, given the risk aversion culture in financial markets that is driven by regulatory and security concerns. Vendor accountability and SLAs can help financial institutions measure and manage their operational risks, which is a regulatory requirement under frameworks such as Basel III.²

Product integrations

The vendor guarantees seamless integration between different commercial products, which in turn can be a major cost-saving factor. Financial institutions rely heavily on trust when conducting a new activity or project. Therefore, they are more likely to adopt a new product or service that integrates with their current offerings if it is offered by a reliable software provider.

Enterprise-grade features

The vendor offers specific features for large corporations, including scalability, security, and compliance. This is particularly relevant for financial institutions operating on a large scale, with stringent requirements of audit and monitoring.

User-friendly experience

Commercial products are often offered with an easy-to-use interface and offer rich documentation and guides. Employees at financial institutions are often users rather than developers; therefore, a user-friendly interface is likely to be more welcomed than a complex one.

On the negative side, commercial software may come with disadvantages such as the following:

Cost

Commercial software licenses can be very expensive and include a variable part that is hard to predict (e.g., support fees, hidden features). For large financial institutions that have critical applications, the risks can easily outweigh the costs.

Bulky products

Many commercial software tools come in a single package that includes a large number of features. Some financial institutions (e.g., FinTech firms) may not need all of these features and end up using only a fraction of what the tool offers.

Vendor lock-in

The more a company relies on commercial tools, the harder it is to switch to other options, including open source. Vendor lock-in gets stronger for factors such as network effect (where product value increases with its user base), and risk aversion (uncertainty from switching to another solution).

Lack of customization

Commercial software is proprietary, meaning that financial clients may not be able to adapt or modify the product to their unique needs or client expectations.

Traditionally, financial institutions have been skewed toward proprietary software because internal control rules, policies, standards, procedures, and IT audit checklists all lay out strict requirements for support agreements. The top financial services software vendors include Microsoft, FIS Global, SAP, Oracle, IBM, and NCR Corporation.

To illustrate why financial institutions have traditionally preferred commercial software, let's take as an example the Oracle Database. If you have ever wondered what type of database systems big banks use for their core operations, the answer is likely to include a mention of Oracle. It is perhaps the most popular enterprise-level database system among financial institutions. Interestingly, [according to the DB-Engines ranking](#), as of 2024, Oracle is considered the most popular database system in the world.

The Oracle Database, a commercial SQL database management system by Oracle Corporation, is predominantly used by financial institutions for online transaction processing (OLTP) purposes. It is used to store and manage core business data encompassing credit, accounts, loans, and transactions.

Over time, new database systems have emerged in the market, and some are gaining traction. However, to date, Oracle remains the gold standard for financial services.

First of all, Oracle has been in the market since 1979; therefore, it is considered a mature and reliable product. Throughout its history, Oracle has consistently [added features to its flagship products](#), placing the company at the forefront of database technology. By offering many features, the Oracle database could fit the many applications, requirements, and use cases that have emerged in the financial sector.

Second, the Oracle Database is flexible because it can run on Windows and various flavors of Unix. Additionally, a variety of Oracle database editions are [available](#), ranging from large-scale editions such as the Oracle Database Enterprise Edition to single-user editions like the Oracle Database Personal Edition. Oracle Database products are available on Oracle hardware as well as being offered by several service providers on premises, on cloud, or as a hybrid cloud installation.

Third, Oracle clients can access Oracle support services through the company's own support team or third-party consultants. This allows fi-

nancial institutions to develop a feeling of trust and security as they have a reliable partner to call when needed.

In conclusion, maturity, stability, and support offer reliability against failures and risks, which is essential for critical applications such as finance. This explains the high popularity of Oracle among financial institutions and in the market in general.

The main alternative to commercial financial software is open source software. Unlike proprietary solutions, open source is typically licensed for free, and its development and maintenance is led by the public and/or by associations such as the Apache Foundation. The most popular type of open source license is the [Apache 2.0 license](#), which allows users to obtain, modify, distribute, sublicense, and patent the software as well as use it for commercial purposes that can include proprietary software.

A large number of open source software options are available. Examples include Linux, PostgreSQL, Firefox, Kubernetes, PyTorch, Apache Spark, Apache Cassandra, Apache Airflow, and many more.

The main advantages of open source include the following:

Cost

Open source software is often available under free licensing. Nevertheless, compared to commercial software, open source might entail indirect costs such as maintenance, upgrades, and feature development costs.

Customization

Because the source code is available to anyone to use and modify, companies can introduce new features that accommodate their business needs.

Community

Open source software tends to have a large community of developers who actively add new features and fix existing bugs. The main

advantage of community contributions is that they aggregate the skills and knowledge of multiple people from different backgrounds and experiences. This, in turn, can guarantee that the final solution is very likely the best in the market.

Transparency

Because the entire codebase is available to the public, open source software tends to offer more transparency into its internal workings, unlike proprietary software, which is a black box.

On the other hand, open source software could face the following drawbacks:

Support

Users can freely report a bug or ask for a new feature in open source software, but this usually takes time as contributors tend to dedicate a variable amount of time to maintain the software. For mission-critical applications such as financial payments, this may not be acceptable.

Documentation

Open source projects may lack up-to-date, detailed documentation and usage instructions.

Complexity

The evolution of open source projects is characterized by frequent updates to address bugs and introduce new functionalities. Consequently, as the codebase grows with additional features, it may become increasingly complex to comprehend.

Compatibility

A major issue with open source software is compatibility with other software applications, which can jeopardize product development and integration efforts.

Security

With open source software, malicious actors and cybercriminals find and exploit code vulnerabilities more easily.

Confidentiality

Open source is based on the idea of community-driven collaboration and knowledge sharing. For financial institutions, sharing their codebase might be impossible if it becomes confidential or provides a competitive advantage that the institution wants to protect.

The use of open source software in financial services has significantly increased in the last few years. According to a [report published by the Fintech Open Source Foundation \(FINOS\)](#), 2022 saw a 43% increase in the number of GitHub repositories with code contributors from the financial services industry. The report indicates that most of the contributions financial institutions make to open source projects are related to web development, cloud and containerization software, AI/ML, and continuous integration/continuous delivery (CI/CD).

Having said all of this, how would you choose between open source and proprietary software?

First, the pros and cons of open source and proprietary software can vary depending on the specific business problem and context; therefore, no single solution can fit all circumstances. In some cases, certain problems can have a well-accepted solution among market participants. For example, if your application requires high standards of security and reliable 24/7 customer support, then commercial software might be the right choice. On the other hand, if you are exploring the possibility of employing AI to build new product features, you might want to use open source libraries. Nowadays, it is common for the same financial institution to use both commercial and open source software.

NOTE

Some commercial software products are based on open source frameworks. These frameworks can either be provided as managed services by vendors or form the foundation for distinct solutions. For instance, PostgreSQL, an open source database management system, serves as the basis for Amazon's Relational Database Service (RDS) for PostgreSQL, a managed service offered by AWS. Conversely, Amazon Redshift, a proprietary data warehousing service, is built on top of PostgreSQL but is a separate product offered by AWS.

Second, consider your budget. If you are a small startup developing cutting-edge financial technology, you will likely be limited in your budget. In this case, using open source might be the best option. On the other hand, if you are a large multinational financial institution with big teams and departments, spending the money on commercial software might be more efficient as it will save the team time and effort when fixing bugs and dealing with security issues. Large corporations prefer to use commercial products as they offer a solution that standardizes internal operations across departments and teams.

Third, the level of technical expertise is a crucial factor. If you are planning on developing software that managers and accountants will use, then unless there is a big IT department, these professionals are likely to have limited expertise in software development to understand and interact with the codebase. In this case, vendor software is likely to cause fewer headaches. On the other hand, if your institution is well-equipped with software engineers who can support and help users, it would be possible to take advantage of the flexibility and developmental freedom that open source software offers.

Fourth, consider the urgency and impact on the competitive advantage of adopting a given software tool. If a financial institution feels behind compared to the market with open source adoption, then it might need to accelerate the adoption process. However, if the company is currently experimenting with open source for potential uses, then it might be possible to do it slowly.

A lot of innovative financial institutions create their own in-house software, mainly in response to customer demand and changing client expectations, which can be challenging to fulfill by relying on third-party software vendors. For example, to boost the time-to-market of its new financial products, the US financial institution JPMorgan Chase developed [Kapital](#), an advanced financial risk management and pricing software system. More recently, JPMorgan Chase leveraged the Python ecosystem to build Athena, a cross-asset platform for trading, risk management, and analytics used internally as well as by external clients. According to a [presentation given by Misha Tselman](#), executive director at JPMorgan Chase, at PyData 2017, Athena has thousands of users, more than 1,500 Python developers contributing to it, and uses over 150,000 Python modules.

Another prominent example is [Aladdin](#), an end-to-end portfolio and investment management platform developed by the largest asset manager in the world, BlackRock. Given its significant success and reliability, the Aladdin platform is no longer only an internal tool; rather, it is significantly influencing the way the financial sector works. Scholars use the term *platform economy* to describe such a scenario, where platforms play a major role in facilitating market activities.

If you are interested in learning more about this topic, I highly recommend the paper by Dirk A. Zetsche, William A. Birdthistle, Douglas W. Arner, and Ross P. Buckley, [“Digital Finance Platforms: Toward a New Regulatory Paradigm”](#), European Banking Institute (EBI) Working Paper Series No. 58 (2020): 273.

Criterion 2: Ease of Use Versus Performance

When choosing software or data technology, a frequent tradeoff arises between complexity and performance. Software technology can be complex if it has a steep learning curve: it may be difficult to understand or master the syntax, many elements must be managed in the code, it is demanding to set up and use, hard to debug, challenging to rerun on other machines, difficult to integrate with other software, and hard to extend with new

features, deploy, or share with others. On the other hand, software is performant if it optimizes dimensions such as execution time, latency, throughput, I/O access, memory usage/footprint, carbon footprint, dependency and package management, scalability, security, or other custom metrics specific to the business operations.

The reason why there is a tradeoff between complexity and performance is that performance optimization often requires low-level interaction and configuration of the technology. Consider, for example, the difference between driving a manual versus an automatic car. To make a more concrete example, let's think about programming languages. A programming language such as C can be considered by many as a challenging language to work with due to its strict syntax requirements, exposure to low-level details, manual garbage collection, static typing, and memory management. Once you learn how to work with languages like C, your performance advantages could be substantial.

On the other hand, programming languages such as Python are well-known for their friendly syntax, usability, automatic garbage collection, and ease of integration. However, a number of features of Python make it less performant in terms of speed than C, such as the reliance on dynamic typing and the multistep compilation process to machine code.

The same logic applies to hardware. For example, engineers can leverage different computer processors and accelerators such as Central Processing Units (CPUs), Graphical Processing Units (GPUs), and Field Programmable Gate Arrays (FPGAs) to perform various tasks.³ The CPU is the computer's main processor, and it excels in performing a wide range of tasks. However, certain applications might require custom optimizations (e.g., high parallelism or low latency), which can be achieved through the use of specialized processors such as a GPU and FPGA. Crucially, using a GPU and FPGA is more complex and requires good knowledge of their internals, cost, and implementation principles.

NOTE

When comparing hardware and software technologies, pay close attention. Not everything is as easy as it looks on the surface. For instance, it's not unusual to come across articles that claim something like a GPU outperforms a CPU, or C# is superior to Python, and so forth. Even though they could have some truth to them, these generalizations have three problems. First, it might be challenging to compare technologies that are created according to different design principles. Second, technologies and the concepts behind their design change throughout time, and as a result, a technology that was once superior to another may not be any longer. Third, depending on the task that they are assessed against, technologies may perform differently. What I recommend in this case is to find a trusted reference from literature that you can rely on to derive a solid foundation for comparison.

Traditionally, financial market participants and researchers have thoroughly tested and benchmarked a number of technologies to certain business and technological requirements. For example, with time-critical or large-scale financial applications such as algorithmic trading, Monte Carlo simulations, asset pricing, and fraud detection, financial institutions might prefer to use performant languages such as C, C++, Rust, and Java.⁴ However, for less critical or small applications or situations where users are not highly skilled in software engineering internals, a more user-friendly language such as Python, R, or Julia might be a better choice.

Low latency is a key differentiator when it comes to performance in financial markets. Generally speaking, the notion of latency is often used to describe the time it takes for a request to travel over a network to its destination and receive a response back from it. In many financial domains, a firm's ability to grasp market opportunities or otherwise lose money is largely dependent on how quickly it reacts to market events. This is particularly the case in electronic trading systems, and more specifically, high-frequency and algorithmic trading.

There are several aspects to latency. First, it may concern the speed at which financial data is harvested and distributed to the many players involved in the trading system. The faster a financial institution can access market data and make trading choices, the more probable it is to capitalize on important short-term opportunities. The second aspect relates to the speed of the order execution flow. The faster an order reaches its destination, the more likely it is that the firm will make profits out of it. A millisecond difference in data access and trade processing time can equate to millions of dollars of gain or loss. For example, when executing an arbitrage strategy to profit from tiny differences in asset prices in two or more markets, the investing firm will have a tiny temporal window before the market reaches parity.

What markets consider "low" latency is not well defined but rather an evolving paradigm. Different financial market segments may operate and make improvements at different orders of time magnitude, such as the second, millisecond (thousandth of a second), microsecond (millionth of a second), and nanosecond (billionth of a second). For example, real-time payments may be considered fast at the second or millisecond level, while high-frequency traders are chasing the microsecond and nanosecond.

Numerous variables can influence latency in the financial markets. For example, a trading firm that speculates on short-term market opportunities will be impacted by the distance between its trading system and the trading venue, the distance between trading venues (in case of arbitrage), the choice of the programming language (e.g., C++ or Rust), hardware

(e.g., CPU or FPGA), the efficiency of the trading program (e.g., time complexity), and cabling (e.g., copper versus fiber versus microwave).

Due to the potential for large returns, low-latency trading firms constantly invest a great deal of resources and effort to accelerate their trading systems. One approach is called *Direct Market Access* (DMA), a trading arrangement that allows traders and investors to place orders directly into the order book of an exchange, bypassing intermediaries such as brokers.

Standardized communication protocols, such as FIX, are essential for DMA, as they allow traders, brokers, and exchanges to exchange trade-related data using a unified language (we discuss FIX in [Chapter 7](#)). DMA usually involves a sophisticated and costly technology setup. Consequently, it is commonly offered as a service by sell-side firms and specialized technology providers.

In addition, to provide markets with the opportunity to gain a speed advantage, stock exchanges started providing trading firms with the option to position their trading servers in close proximity to, or even within the same data center as, the exchange servers.

This strategy, called *colocation*, allows trading firms to access financial data through data feeds that stream data right from the source as soon as it appears in the exchange server. Colocation has [raised some issues of fairness in financial markets](#), as the average investor or market maker will get the information at a later date, when they will have less advantage to react.

Another direction toward low latency in financial markets relies on the use of specialized hardware such as a field-programmable gate array (FPGA). An FPGA is a configurable type of integrated circuit that can be programmed after manufacturing to suit whatever purpose and needs you want. As such, FPGAs have been widely adopted by financial markets to build systems that react to market events and process orders in nanoseconds, as well as to perform complex/parallel computations efficiently.

Criterion 3: Cloud Versus On Premises

In today's technological landscape, a major and critical technological choice often emerges between cloud-based and on-premises infrastructures. In an on-premises server infrastructure, the firm owns, controls, and maintains a group of servers (often called commodity servers) for its storage and computing purposes. Alternatively, a firm can use the cloud to host or lease servers at a third-party cloud provider or use the vendor's managed servers and services directly.

There are pros and cons to using either on premises or the cloud, and making the right decision may be quite challenging. Generally speaking, on premises is viewed by many as the traditional (or legacy) way of managing server infrastructure, while the cloud represents the modern and user-friendly approach. This, however, doesn't mean that on premises is bad and the cloud is good. Let's explore in more detail the main differences and features of both solutions.

On premises

In an on-premises setting, the financial institution is fully responsible for its servers and data infrastructure. This means that the institution owns, configures, and manages its servers, computing environments, networking, security, scalability policies, logs, and user access.

As you might have guessed, the main advantage of this infrastructure choice is that you have full control of your digital assets. If managed properly, an on-premises infrastructure can guarantee maximum security. Having all your data and software reside within your organization's premises provides a feeling of safety and peace. Security is a major concern for highly regulated businesses like finance; hence, on premises is a very typical choice.

Nevertheless, the on-premises approach has a number of potential downsides. Cost is perhaps the main issue. To maintain an on-premises infrastructure, institutions need to have a dedicated IT department responsible for software and hardware maintenance, availability, updates, security, license purchases, and user support. Such costs might rise exponen-

tially, especially in large institutions or in the presence of poor management and usage practices.

TIP

Keep in mind that cost is not always a downside to on-premises solutions. In some cases, it might be the best cost-efficient solution if configured and managed with high quality standards as well as software-aware and hardware-aware best practices. For example, in 2023, [X \(formerly known as Twitter\) announced that it implemented a cloud exit strategy](#) by moving data and workload out from the cloud and onto its own on-premises servers. The move resulted in a 60% decrease in cloud data storage size and a 75% decrease in cloud data processing costs.

The second downside is scalability. In an on-premises setting, institutions have a certain number of servers that they manage. As long as the infrastructure can handle the load, there should be no problems. However, if the load increases much more than the current server infrastructure can handle, then either the system will experience downtime issues or more servers need to be added. Adding more servers can be costly and time-consuming; and once added, they can't simply be removed and returned. One solution to this issue is to overprovision by having extra server capacity all the time. However, this strategy can lead to extra costs due to unused resources.

The third drawback is that institutions may find themselves spending less time on product and feature development and more time on server maintenance and time-consuming technical, bureaucratic processes. Large financial institutions may tolerate this; however, for smaller institutions such as FinTech firms, the burden may not be sustainable.

As far as the financial sector is concerned, it is hard to avoid mentioning *mainframes*. In a few words, a mainframe is a highly performant, scalable, and resilient computer endowed with large amounts of memory and processing power. The main advantage of mainframe computers is their ability to handle massive volumes of simultaneous transactions and requests while at the same time ensuring reliability, security, availability, high throughput, and low latency.

This feature makes mainframes well-suited for core financial applications such as banking, customer order processing, payment processing, and various mission-critical tasks that process billions of transactions on a daily basis. If you have ever interacted with an automated teller machine (ATM) or point-of-sale terminal at a retail shop, your request likely involved a mainframe at some point.

The market pioneer in mainframes is [IBM](#), which has been producing mainframe computers since 1952. IBM mainframes have evolved over time to meet the diverse requirements of resilience, security, and performance. The primary mainframe system offered by IBM is the IBM Z System, which includes various generations such as the IBM z15, IBM z14, and IBM z13. Complementing these mainframes are a range of related IBM offerings, including IBM's proprietary operating system, known as IBM z/OS; IBM's transaction processing system, known as the Customer Information Control System (CICS); IBM's relational database management system DB2; and the middleware communication system IBM MQ. This integrated and secure ecosystem has enabled IBM to maintain a significant market share within the financial sector, particularly for payment and transaction processing.

Cloud computing

A major alternative and competitor of the on-premises model is the cloud. In this book, we define the cloud as:

A general-purpose technology that enables the development and delivery of computing and storage services over a networked system.

In this definition, the term *general-purpose technology* refers to a technology whose adoption and impact span many sectors within the economy, substantially impacting preexisting social and economic structures. The best examples of general-purpose technologies are electricity, the internet, and artificial intelligence. The cloud is already a general-purpose technology, given its wide range of applications and adoption in businesses, governments, hospitals, research centers, financial institutions, educational institutions, the military, and many more.⁵

The most important thing about the cloud is how it's delivered, not just the services it delivers. Many of the underlying technologies powering the cloud already exist and are utilized in various applications. This includes, for example, software and hardware virtualization technologies, multi-core technologies, networking (network computing, software-defined networking, software-defined security, WAN), data centers, database management systems, file storage systems, open source tools, machine learning, and DevOps, to name a few. However, with the cloud, such technologies have been abstracted and architected in a novel way that allows for the delivery of scalable, highly available, managed, and pay-as-you-go services through a networked system. The public internet is the primary delivery network, but it is also possible to establish a private cloud that operates on premises or in a data center managed by a cloud provider. In the next section, we will explore in detail the differences among public, private, and hybrid clouds.

Cloud products can be classified into three main categories:

Software-as-a-service (SaaS)

SaaS products are delivered to customers via the internet and are entirely managed and maintained by the cloud provider. With a SaaS solution, users can easily get started with the product, often instantly upon confirming a subscription plan, and they do not need to worry about configuration, software licensing, installation, or upgrades. Users might still need to confirm who can access their

SaaS and what privileges they have. Examples of SaaS products include Gmail, GDrive, BigQuery, Dropbox, Snowflake, and many more. Sometimes, SaaS solutions get assigned more precise names, such as database-as-a-service and AI-as-a-service.

Platforms-as-a-service (PaaS)

PaaS provides online platforms for developing applications via APIs and operating system services. Consumers of PaaS control the settings of the hosting environment, the application, and access policies but not the infrastructure, storage, and networking. SaaS apps are typical users of PaaS. Examples of PaaS include the Google App Engine and DigitalOcean App Platform.

Infrastructure-as-a-service (IaaS)

IaaS is the lowest level of cloud offering and provides raw physical resources (CPU, RAM, storage, and networking) as a service, with the user having nearly complete control over the configuration of the instances. The majority of SaaS and PaaS apps are based on IaaS. Examples of IaaS include AWS EC2 and Google Compute Engine.

NOTE

The cloud market is growing into a complex landscape of providers, services, and products. It is important to keep in mind that there are differences between a cloud provider and a cloud-based service provider. Cloud providers own and provide a large number of data centers distributed all over the world. These providers are called [hyperscalers](#) and include major players such as AWS, Microsoft, and Google. On the other hand, there exist a large number of cloud-based service providers that develop and offer services through the cloud. The best example is Snowflake, which offers a cloud-based data warehouse solution that runs on AWS, Google, or Microsoft.

In the financial sector, cloud computing has drawn a lot of interest, especially as a means to boost innovation and save expenses. According to a 2022 [report by McKinsey](#), the adoption of the cloud is on most financial institutions' agendas. A large number of strategic partnerships have been

announced between major cloud providers/service providers and financial institutions. For example, BlackRock, the largest asset manager in the world, has [partnered with Snowflake](#), a cloud data warehouse provider, to offer a cloud-based version of its investment platform Aladdin; Goldman Sachs [established a collaboration with AWS](#) to create cloud-based data management and analytics solutions for the financial sector; JPMorgan Chase [established Fusion](#), and State Street [created Alpha](#), both cloud-based integrated platforms for investment and financial data management.

Migrating to the cloud can bring a large number of benefits for financial institutions. These include the following:

- Quicker time to market, facilitated by the ease with which resources can be provisioned and managed.
- More innovation as developers can test and experiment with new ideas without having to buy hardware and incur unnecessary costs.
- Continuous access to novel technologies developed by cloud providers, which would otherwise be very costly to develop in-house.
- Cost savings due to the pay-as-you-go model, where users pay only for what they use. This shifts the cost structure from fixed IT capital expenditure to variable operating costs based on demand.
- Scalability, where users can scale resources up and down based on their current and planned needs without having to purchase physical servers.
- Better collaboration as the cloud allows for easy and quick sharing of resources, files, and proof-of-concept demonstrations.
- Advanced security and data protection features such as Identity and Access Management (IAM), backups, encryption, centralized management, and audit logs.
- Cloud providers offer multiregion options for data storage and resource provisioning. This increases operational resilience as it reinforces availability. Such a feature is quite crucial for financial applications. For example, for a digital banking firm that is *branchless*, online services must always be on and available at any time of the day.
- Compliance: cloud computing's multiregion feature allows financial institutions to pin their customer data to a specific region to meet the

data privacy requirements of that location.

Logically, as with most technologies, cloud computing has its drawbacks. Examples include the following:

- Internet access, which raises the risk of access loss in case of internet or service downtime
- Lack of full control over the underlying infrastructure and data
- Security and privacy concerns
- Regulatory constraints
- Integration challenges with existing systems
- Vendor lock-in
- Unforeseen costs and consumption patterns that defy the original migration goals

One of the major perceptions about cloud technologies is their lower degree of security compared to hosting data in-house on an on-premises infrastructure. The fundamental justification for this opinion is that shifting data to a third-party location entails giving up complete ownership and control of that data as well as opening it to the public since you are likely to access it via the internet.

Even though such concerns are rational, cloud computing isn't that insecure. Cloud service providers make significant investments in cloud security, regularly hire security professionals, and use cutting-edge security measures. For example, in August 2023, Google [blocked a massive denial-of-service attack \(DoS\) on its infrastructure](#) that reached 398 million requests per second (rps). To put things in perspective, this two-minute attack generated more requests than the total amount of Wikipedia page views during the entire month of September 2023.

Nevertheless, cloud security should be carefully examined and assessed before making any major decision. I highly recommend checking the cloud vendor certifications and compliance with the different security regulations and standards for different sectors. Examples of security certifications include the following:

[ISO/IEC 27001:2022](#)

Information security, cybersecurity, and privacy protection

[ISO/IEC 27017:2015](#)

Guidelines for information security controls applicable to the provision and use of cloud services

[ISO/IEC 27018:2019](#)

Code of practice for protection of personally identifiable information (PII) in public clouds acting as PII processors

Cloud providers often include a comprehensive list of their compliance offerings grouped by country, sector, region, and more. Examples include

the following:

- Google [compliance offerings for financial services](#)
- AWS [compliance programs and certifications](#)
- Microsoft [compliance offerings](#)

Additionally, most cloud providers have offerings tailored to the needs of the financial sector. Examples include IBM's [Cloud for Financial Services](#), Google's [Cloud for Financial Services](#), AWS's [Cloud Solutions for Financial Services](#), [Microsoft Cloud for Financial Services](#), and [Snowflake's AI Data Cloud for Financial Services](#). What distinguishes these cloud offerings is the focus on data protection, privacy, security, fraud detection, and several other features that financial institutions require.

To evaluate the suitability of cloud services for highly confidential financial data, Ilya Epshteyn from AWS [proposed a framework consisting of five key factors](#), which I recommend as an example of a general guideline.

When considering a cloud migration decision, keep in mind that it must be done through a cloud migration strategy that matches your institution's goals and requirements with what the cloud offers. Otherwise, migration efforts might easily run into unexpected issues such as excessive costs, technological limitations, security and compliance issues, scalability, and control.

A full treatment of cloud migration strategies is beyond the scope of the book.⁶ However, from a high-level view, a cloud strategy is, in the first place, an economic strategy.⁷ If moving to the cloud is going to increase your costs or not have a major impact on your costs or revenues, then it might not be worth it. Consider calculating your cloud strategy's [return on investment \(ROI\)](#) and proceed gradually based on your analysis. For example, it might initially seem more appealing to migrate your analytical data to a cloud-based managed data warehouse service for machine learning purposes. Successively, other types of data can be migrated to a data lake, NoSQL database, and other data stores. Should this be benefi-

cial, you might consider migrating your virtual machines and gradually the rest (or a significant part) of your infrastructure.⁸

Second, consider analyzing the potential impact of cloud migration on your business models and the value creation logic. For example, businesses need to get used to the idea of web-based services and delivery mechanisms, pay-per-use expenditure models, managed services, and the shared responsibility model.⁹

Third, consider the technological limitations of the cloud and their impact on your business needs. Cloud technologies are very powerful, but they have limitations. Some of these limitations are inherently derived from the technology itself (e.g., the connection limit in a PostgreSQL database) or cloud-related limitations, such as the shared resources model of the cloud, which might limit the amount of RAM or CPU dedicated to a virtual machine.

Fourth, your strategy must carefully assess your company's data governance and compliance requirements to ensure you keep your system resilient and secure. Cloud services are user-friendly and relatively easy to use. But with such ease comes a challenge, which is the risk of misusing cloud services and creating chaotic architectures. This, in turn, might impact the level of control and quality of the data and infrastructure you host in the cloud.

Criterion 4: Public Versus Private Versus Hybrid Cloud

With the adoption of cloud computing worldwide, three cloud models have emerged: public, private, and hybrid. In this section, we will explore the main features of each model and illustrate their relevance to financial institutions.

Public cloud

In a public cloud model, the underlying infrastructure (compute, storage, networks, etc.) is owned and managed by a third-party vendor and acces-

sible by users over the public internet. Examples of public cloud providers include Amazon Web Services, Google Cloud, Microsoft Azure, IBM Cloud, and Oracle. The public cloud is the most popular model and is often the default choice.

A key feature of the public cloud is *multitenancy*, meaning that cloud service users share the underlying infrastructure while remaining isolated through virtualized environments. Multitenancy allows the cloud vendor to optimize the use of resources and reduce costs.

Due to multitenancy, the public cloud model advocates the *shared responsibility principle*, wherein both the cloud provider and users assume distinct yet overlapping responsibilities to safeguard the security of services and data hosted in the public cloud. The cloud provider oversees physical infrastructure security and maintains a logical separation between client data and resources. Simultaneously, clients are responsible for ensuring application-level security, such as user access and permissions, data encryption, backups, and multifactor authentication.

The main advantages of the public cloud include the following:

Simplicity

Public cloud providers focus on making their services user-friendly to cater to the diverse needs of all sectors and clients they serve.

Flexible pricing

Public cloud users can pay on demand while also capitalizing on discounts offered for committed usage, where they reserve a resource for a predetermined duration.

Scalability

Cloud providers strategically plan and provision their infrastructure to accommodate variable resource needs, ensuring clients can access the necessary resources whenever needed.

Minimal configuration and maintenance burden

The cloud provider manages and maintains the data centers hosting the physical servers.

On the negative side, the public cloud may not be ideal for the following reasons:

- Data confidentiality and security, as data is stored at a third-party location and accessed through the public internet
- Excessive costs, which may arise with large-scale applications, miscalculated resource needs, or unexpected spikes in demand
- Limited infrastructure control

Given their security and confidentiality requirements, financial institutions looking to adopt the cloud might find the public cloud the most risky option. However, the public cloud might be a good solution if used for less confidential types of data, e.g., machine learning. Among financial institutions, FinTech firms are major public cloud users, given the extra features it provides in terms of products and flexibility, which are ideal for innovation.

The most critical security threat when using public cloud services is [cloud misconfiguration](#). Since the public cloud is built on the shared responsibility principle, customers may incorrectly set access to their resources, leaving them vulnerable to cybercriminals' attacks. For example, a common misconfiguration happens when a resource is accidentally made public or given excessive permissions that can be easily exploited to gain access to sensitive data.

In 2019, over 100,000 social security numbers, credit card details, and client financial information were stolen as a consequence of a cloud [data breach at the US financial company Capital One](#). The breach was possible thanks to a misconfiguration in permission settings that allowed one resource to access data stored in the storage service AWS S3 that Capital One used to store its client data. The misconfigured resource was attacked and used to gain access to client data in S3.

Cloud providers have [implemented measures to help users detect and mitigate cloud misconfiguration issues](#). Nevertheless, if you use a public cloud service, I highly recommend that you assign primary importance to cloud misconfiguration and vulnerability risks.

Private cloud

In the private cloud model, an organization has a dedicated cloud infrastructure that is not shared with others. The dedicated infrastructure can be owned and hosted by the organization itself, owned by the organization and hosted at a third-party vendor, or owned by the vendor and rented by the organization. The rise of the private cloud primarily stemmed from large corporations with data centers seeking to replicate the public cloud model internally.

The private cloud's main advantages are control and security, which makes it an ideal solution for many regulated industries, such as financial services. If a financial institution builds and deploys a private cloud on its data centers, then security is the full responsibility of the institution. If a

private cloud is deployed at a third-party data center, then the servers' physical security is the cloud provider's responsibility. In both cases, the security of the data and user access management is the responsibility of the institution.

As a main downside of the private cloud, it is considerably harder to set up and deploy, especially if managed fully by the institution in-house. The private cloud might require significant up-front investment in infrastructure and IT talent and lead to ongoing costs in terms of maintenance and personnel.

The private cloud has received considerable attention among financial institutions, which tend to favor security and compliance over other features. Large financial institutions already owning big data centers might benefit greatly from switching or adapting to a private cloud setting. For example, Bank of America invested heavily in building an internal private cloud, which, according to a [*Business Insider* article](#), helped save \$2 billion in annual infrastructure expenditure and reduced the number of servers from 200,000 to 70,000 and its data centers from 60 to 23. This reduction was possible through the use of virtualization technologies, which led to a reduced need for servers.

As a viable and more practical alternative, cloud providers offer the option to deploy a [*Virtual Private Cloud \(VPC\)*](#), which is deployed within a public cloud infrastructure. A VPC offers the best of both worlds: the private cloud's security and reliability and the public cloud's scalability and convenience. A VPC can allow a financial institution to isolate compute resources and network traffic for customers' workloads that involve highly confidential data.

Hybrid cloud

The hybrid cloud model combines public and private clouds to take advantage of both. In a hybrid setting, a private cloud is often used for operations that involve confidential data, such as client transactions and financial reporting, while the public cloud is used for high-volume and less sensitive operations, such as log analysis, machine learning, and web-

based applications. The public cloud can also be dedicated to handle occasional spikes in workload from the applications running in a private cloud, a technique known as *cloud bursting*.

A hybrid cloud can be an ideal solution for financial institutions, offering the flexibility and scalability of the public cloud while ensuring regulatory compliance and security through the private cloud. Moreover, the hybrid cloud model allows for cost optimization by allocating resources and workload between public and private cloud based on demand and circumstances.

The main disadvantages of a hybrid cloud are the cost involved in building and maintaining it and the additional operational complexity that derives from the need to coordinate and orchestrate two clouds.

Alternatively, a company can build a hybrid cloud via a combination of a VPC and public cloud. In this case, the maintenance burden of the private cloud is lower as the cloud vendor maintains the underlying infrastructure.

Criterion 5: Single Versus Multi-Cloud

Multi-cloud is a cloud strategy whereby an organization uses services from multiple cloud providers.

Starting with a single cloud provider is often the default and simplest solution. This is because most cloud providers offer similar products, e.g., virtual machines, managed SQL databases, file storage, and access management. However, as new business requirements and features get added, limitations might emerge in the cloud offerings of the trusted cloud provider. A company that uses cloud provider A might discover that a database service from provider B aligns more closely with the technical and cost prerequisites of the business, thus making it a more favorable option.

NOTE

The choice of a public cloud provider may depend on factors that are not necessarily technology related. For example, among large financial institutions, it is common to see a preference for Microsoft Azure cloud services. This is due to the long-standing relationship of trust and security that many companies have developed with Microsoft. When formulating a cloud migration strategy, consider an approach that encompasses trust and relationships, as well as considerations regarding offerings, technology, features, and pricing.

A multi-cloud strategy can also be more economical. For example, Google's warehouse solution BigQuery charges the user based on the amount of bytes that their queries fetch. On the other hand, Snowflake's data warehouse solution charges based on the duration of the query. If your queries are data-intensive but fast, then Snowflake might be more economical, while if your queries take some time but don't fetch large amounts of data, then BigQuery might be more appealing.

Cloud pricing can be tricky and hard to understand. Before making your choice, consider carefully all costs involved in purchasing a cloud service, particularly managed database services. A large number of factors might impact costs other than the single-unit on-demand price reported on the vendor's website. Even though the individual cost might seem low, if you can't predict your scaling needs, costs can easily accumulate to a large sum. Additionally, pay careful attention if you decide to use cloud cost benchmarking, as it might be misleading or biased if you don't understand the settings or context in which they were conducted.

In some cases, multi-cloud can be a feature of a cloud-based product itself. For example, Snowflake cloud-based data warehousing relies on the principle of separation between storage and computing. This means that data can be stored and managed independently of the compute resources that are required to interact with the data. Snowflake allows clients to choose the cloud platform (Azure, AWS, or Google) on which to deploy the Snowflake service. This strategy makes it more appealing for clients to use Snowflake, as their data doesn't need to leave their cloud infrastructure of choice.

A multi-cloud strategy can mitigate the risk of vendor lock-in. The term *vendor lock-in* refers to a situation where the cost and effort needed to switch from one cloud provider to another is so high that the client is essentially stuck with the current cloud provider. Vendor lock-in may be disadvantageous in case the provider makes changes to their services, increases price, or incurs downtime issues. By adopting a multi-cloud strategy, you maintain your flexibility to adapt to changes introduced by a given provider and make use of each provider's best features.

The main drawback of the multi-cloud strategy is the additional management and operational burden needed to maintain the resources hosted on different cloud platforms. Using two clouds means securing both and making sure access is coordinated and managed properly. Establishing a secure and private connection between multiple clouds can be both complex and expensive, often requiring intricate VPN configurations or other advanced networking solutions. Additionally, consistency and reliability checks must be implemented if data needs to be moved between clouds. Moreover, if resources hosted on different clouds become tightly coupled, this may easily increase costs, impact performance, and jeopardize maintainability.

In 2021, Wells Fargo, a US multinational financial services company, [announced a new digital infrastructure strategy](#) that uses services from two public cloud providers as well as third-party-owned data centers for private cloud and traditional hosting services.

Wells Fargo's strategy relies predominantly on Microsoft Azure's public cloud to drive innovation across all departments and ensure a secure and trusted environment for strategic business workloads. It uses Azure Cloud as the main foundation for most day-to-day data and analytical needs and to empower employee collaboration. However, it also leverages the Google Cloud platform for more advanced data and analytical workloads, such as artificial intelligence, and to develop personalized customer solutions.

Furthermore, Wells Fargo's digital strategy complements its public cloud infrastructure with third-party-owned data centers, leveraging private cloud and traditional hosting services to create a secure, reliable, and flexible digital foundation.

In general, Wells Fargo's hybrid multi-cloud architecture is viewed as a promising future trend in cloud computing since it provides the greatest flexibility for meeting business demands in terms of performance, security, cost, scalability, and innovation. To explore this trend in detail, I highly recommend the excellent work of Paul Zikopoulos, Christopher Bienko, Chris Backer, Chris Konarski, and Sai Vennam, [*Cloud Without Compromise*](#) (O'Reilly, 2021).

Criterion 6: Monolithic Versus Modular Codebase

A major decision that companies have to make when building a data infrastructure is the organizational style of the codebase and application assets. Codebase organization style is not concerned with how the software works but rather how it is structured, linked, and deployed. Code structure can be crucial in determining the application's readability, scalability, and reliability. To this end, two main codebase styles are often pro-

posed, primarily *monolithic* and *modular*, which I briefly illustrate in this section.

Monolith architecture

In a monolith architecture, the codebase is typically organized in a single location and is characterized by tight coupling between its constituent elements. As a consequence, when making a change to a monolith, it is very likely that you will have to deploy the entire application and not just the part that you changed. In its simplest state, you can think of a monolith as a single GitHub repository that stores the entire application codebase.

It is also possible to have a distributed monolith, which organizes the codebase in multiple locations, but due to tight coupling, a change in one place requires changes and redeployment of all or several of the other locations.

Monolith architectures may provide a number of advantages, such as the following:

- Ease of deployment, as you don't deal with the pitfalls of distributed systems
- High levels of cohesion and consistency
- Simpler development workflow, as all components of the code are visible to the developer
- Easier to monitor
- Easier to test (e.g., end-to-end)
- High throughput and performance (no need to communicate with many other services)
- Simplifies code reusability (all code is in one place; use what you need!)

The monolith architecture may be ideal for simple and lightweight financial applications, both for developers and for the application's extensibility and performance.

Nevertheless, monolith architectures may lead to a very complex and hard-to-predict application codebase, making it quite challenging to scale,

extend, understand, and debug. The more components and dependencies an application has, the harder it is to understand the impact of a local change on the system's overall behavior.

NOTE

Monolith architectures tend to be seen as a bad thing by default. This, however, is not necessarily true. A monolith architecture is a choice. It can be pretty good in some contexts but should be treated carefully and on a case-by-case basis to avoid its pitfalls. In 2023, Amazon Prime Video [decided to switch back to a monolith architecture from a distributed microservices architecture](#). The move to a monolith architecture helped achieve higher scalability and resilience, and reduced infrastructure costs by 90%.

Modular architecture

In a modular architecture, an application is split into smaller modules that can be developed, deployed, tested, and scaled separately. Different teams may own different modules and manage the resource and feature requirements of each separately.

The most popular modular architecture pattern is the *microservices*, a term used to describe small, autonomous applications that work together to achieve a common goal. In many cases, a microservice architecture is produced from refactoring a complex monolith codebase that has reached the limits of scalability and performance. In [Chapter 11](#), we will talk about microservice workflows, where you will learn more about microservice-related concepts and design patterns.

TIP

If you are considering a refactoring project of your monolith to a microservice architecture, make sure you plan the migration ahead of time and produce the necessary software architecture metrics and evaluations. A number of best practices exist in this regard. For example, [software architecture evaluation techniques](#) can be used to assess the quality and reliability requirements of a proposed/chosen software architecture. Moreover, tools such as the Modularity Maturity Index (MMI), fitness functions, and [software metrics](#) can be used to assess which components of the system need to be refactored, replaced, or left as they are.

Summary

This chapter provided a general overview of the technical implementation aspects of financial data engineering. It covered the financial data engineering lifecycle as a framework for organizing the various layers of a financial data infrastructure: ingestion, storage, transformation and delivery, and monitoring.

Following that, the chapter outlined a set of six criteria intended to assist financial institutions in making informed decisions when evaluating technological alternatives for the FDEL stack. These criteria serve as a general guideline applicable to all layers of the FDEL. In the next chapters, you will learn about additional criteria specific to each layer within the FDEL.

The next four chapters will cover each of the four FDEL layers in depth to provide more technical details. More specifically, [Chapter 7](#) will cover the ingestion layer, [Chapter 8](#) the storage layer, [Chapter 9](#) the transformation and delivery layer, and [Chapter 10](#) the monitoring layer.

- 1** Layered architectures are quite common in software engineering. Their main advantages include simplicity, maintainability, familiarity, and cost. To learn more about this topic, I recommend Mark Richards and Neal Ford's [Fundamentals of Software Architecture: An Engineering Approach](#) (O'Reilly, 2020).
- 2** According to the [Basel framework](#), operational risk is defined as “the risk of loss resulting from inadequate or failed internal processes, people and systems or

from external events.”

- 3** For a high-level introduction to the CPU, GPU, and FPGA, I recommend Intel’s article [“Compare Benefits of CPUs, GPUs, and FPGAs for Different oneAPI Compute Workloads”](#).
- 4** For a benchmark study, I recommend S. Borağan Aruoba and Jesús Fernández-Villaverde’s [“A Comparison of Programming Languages in Macroeconomics”](#), *Journal of Economic Dynamics and Control* 58 (September 2015): 265-273.
- 5** For a detailed study on the cloud as a general-purpose technology, I recommend Federico Etro’s [“The Economic Impact of Cloud Computing on Business Creation, Employment and Output in Europe”](#), *Review of Business and Economics* no. 2 (January 2009): 179–208.
- 6** For a good introduction, see Jamil Mina, Armin Warda, Rafael Marins, and Russ Miles’ [Digitalization of Financial Services in the Age of Cloud](#) (O’Reilly, 2022).
- 7** An excellent reference on cloud economics is Joe Weinman’s *Clouconomics: The Business Value of Cloud Computing*, + Website (Wiley Online Library, 2012).
- 8** To learn more about cloud migration strategies, see [“What Is a Cloud Migration Strategy?”](#) by VMware.
- 9** For an interesting read on this topic, I recommend Thomas Boillat and Christine Legner’s [“From On-Premise Software to Cloud Services: The Impact of Cloud Computing on Enterprise Software Vendors’ Business Models”](#), *Journal of Theoretical and Applied Electronic Commerce Research* 8, no. 3 (December 2013): 39–58.